



Politecnico di Torino

Communications Systems

Assignment 2 Report

Mahyar Onsori 309823

November 2022

In this assignment, we focus on error correction, pseudorandom binary sequences and sync marker detection.

Exercise 1

First, we generate same G and H as the previous assignment. Then we produce the binary information vectors and codebook and compute the distance profile and minimum distance for the codebook.

We know that the codeword error probability for error correction hard is obtained through this formula:

$$P_W(e) = P(C_R \neq C_T) = 1 - P(C_R = C_T) = 1 - P(e \in D(0))$$

$$\leq 1 - P(e \in S(0, t)) = 1 - \sum_{i=0}^t \binom{M}{i} P^i \cdot (1-p)^{M-i}$$

For implementing the equation, we need P . But in the assignment we have $\frac{E_b}{N_0}$ which is in dB. We have to convert the range of 0:10 dB into probability. At first we have to convert $\frac{E_b}{N_0}$ back from dB.

$$\frac{E_b}{N_0} [dB] = 10 \cdot \log_{10} \frac{E_b}{N_0} \Rightarrow \frac{E_b}{N_0} = 10^{\frac{\frac{E_b}{N_0} [dB]}{10}}$$

Also we know that in a 2-PSK constellation, probability of bit error rate is $P_b(e) = \frac{1}{2} \operatorname{erfc} \sqrt{\frac{E_b}{N_0}}$. But this is not the p in the analytical formula for codeword error rate. This is for an uncoded transmission. But now, we map a k -bit vector with energy E_b to a M -bit vector with energy E_b^* . Therefore we have:

$$P = \frac{1}{2} \operatorname{erfc} \sqrt{\frac{E_b^*}{N_0}} = \frac{1}{2} \operatorname{erfc} \sqrt{\frac{k}{M} \frac{E_b}{N_0}}$$

Now we have the probability and true value of $\frac{E_b}{N_0}$ and we compute the codeword error rate for error correction hard. The results are given in the Table 1-1 and are shown in Figure 1-1 .

$\frac{E_b}{N_0} [dB]$	Codeword error rate
0	3.702×10^{-1}
1	2.824×10^{-1}
2	1.990×10^{-1}
3	1.268×10^{-1}
4	7.120×10^{-2}
5	3.416×10^{-2}
6	1.351×10^{-2}
7	4.217×10^{-3}
8	9.855×10^{-4}
9	1.615×10^{-4}
10	1.710×10^{-5}

Table 1-1: Codeword error rate of error correction hard for given values of E_b/N_0 using analytical formula

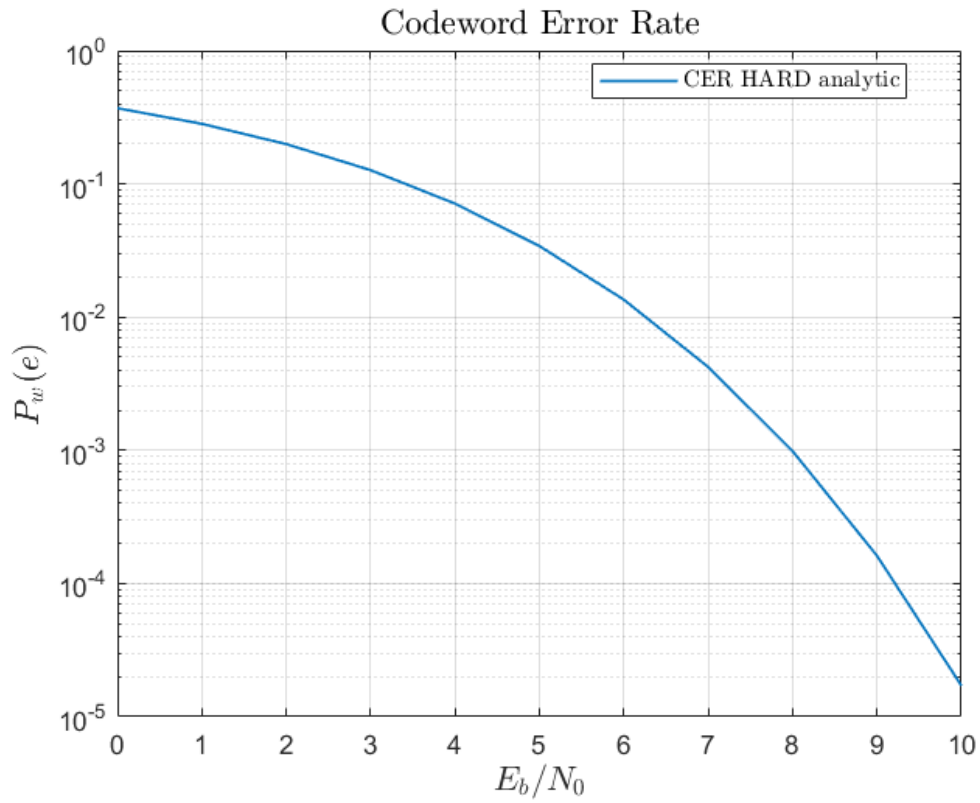


Figure 1-1: Codeword error rate for error correction hard using simulation

For simulation part, we must do syndrome decoding. But first, we want to explore what syndrome decoding is.

At the receiver side, we compute the syndrome $s = y.H$ but we know that s only depends on the error vector e because:

$$y = c_T + e, \quad s = y.H = (c_T + e).H = c_T.H + e.H = e.H$$

Knowing this, we can create a Look Up Table (LUT) which displays the relationship between a pair of syndrome and error vector. The all-zero error vector always is paired with all-zero syndrome. For errors with weight ≥ 1 which are inside the sphere (and therefore are corrected) we must compute the dot product of error vector and parity check matrix. In this case $t = 1$ and we have to build a Look Up Table for error vectors with weight ≤ 1 . Consider error vector $e = 10000000$:

$H =$	$e =$
0110	1
1011	0
1101	0
1110	0
1000	0
0100	0
0010	0
0001	0

If we multiply them, we get $s = 0110$. If we consider error vector $e = 01000000$ we get $s = 1011$. We can observe H is repeated in the LUT. So LUT looks like this:

s	e
0000	00000000
0110	10000000
1011	01000000
1101	00100000
1110	00010000
1000	00001000
0100	00000100
0010	00000010
0001	00000001

Now we can compute the error rate for error correction hard using simulation. For this task, at first, we generate a binary information vector v and encode it with G to obtain c . Now we use 2-PSK labeling and produce c' which is generated through:

$$c' = \begin{cases} -1, & c = 0 \\ 1, & c = 1 \end{cases}$$

Now we send the c' through an AWGN channel with zero mean and variance σ^2 . To determine the σ for using in MATLAB command, we use this formula:

$$\sigma = \sqrt{\frac{1}{2 \frac{k}{M} \frac{E_b}{N_0}}}$$

After sending through AWGN channel, we obtain r and for each bit of r , we decide that the bit was originally 0 or 1 looking at its sign and we obtain y . Now we compute the syndrome and look at the LUT. If the syndrome is not in the LUT then we label the received codeword as wrong. If the syndrome is inside the LUT then we recover e and we sum it with y to build the received codeword c_R and we check it with the transmitted codeword, if they are not equal the codeword is wrong.

The results for simulating the codeword error probability for error correction hard are given in Table 1-2 and are shown in Figure 1-2.

$\frac{E_b}{N_0} [dB]$	Codeword error rate
0	3.691×10^{-1}
1	2.831×10^{-1}
2	2.003×10^{-1}
3	1.308×10^{-1}
4	7.230×10^{-2}
5	3.490×10^{-2}
6	1.424×10^{-2}
7	4.214×10^{-3}

Table 1-2: Codeword error rate of error correction hard for given values of E_b/N_0 using simulation

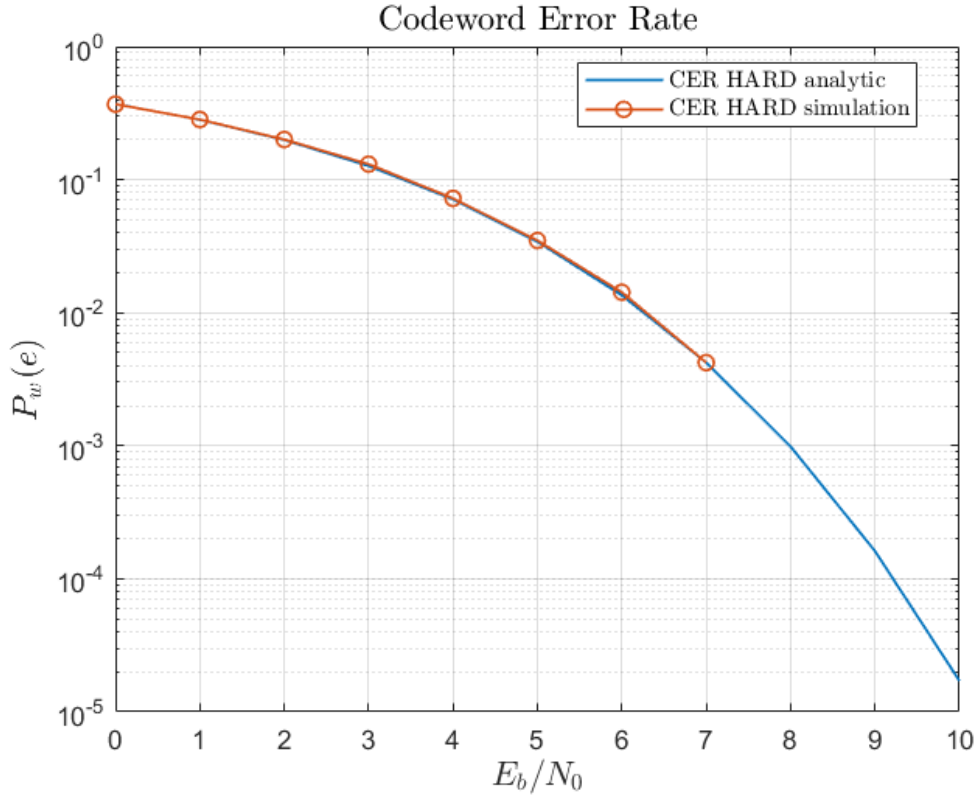


Figure 1-2: Codeword error rate for error correction hard using both analytical formula and simulation

Now we turn to soft correction part. At first, we introduce the expressions for union bound and asymptotic approximation.

Union bound for soft correction codeword error rate is obtained through this equation:

$$P_W(e) \leq \sum_{i=1}^M \frac{1}{2} A_i \operatorname{erfc} \sqrt{\frac{k}{M} \cdot i \cdot \frac{E_b}{N_0}}$$

Asymptotically we have:

$$P_W(e) \cong \sum_{i=1}^M \frac{1}{2} A_{\min} \operatorname{erfc} \sqrt{\frac{k}{M} \cdot d_{\min} \cdot \frac{E_b}{N_0}}$$

Using the multiplicity values and minimum distance we computed before, we calculate the values for these two expressions and display them in Table 1-3 and Figure 1-3, respectively.

$\frac{E_b}{N_0} [dB]$	Union Bound	Asymptotical approximation
0	3.389×10^{-1}	1.249×10^{-1}
1	1.906×10^{-1}	7.795×10^{-2}
2	9.534×10^{-2}	4.382×10^{-2}
3	4.144×10^{-2}	2.163×10^{-2}
4	1.521×10^{-2}	9.073×10^{-3}
5	4.560×10^{-3}	3.104×10^{-3}
6	1.069×10^{-3}	8.227×10^{-4}
7	1.858×10^{-4}	1.582×10^{-4}
8	2.217×10^{-5}	2.035×10^{-5}
9	1.640×10^{-6}	1.579×10^{-6}
10	6.570×10^{-8}	6.481×10^{-8}

Table 1-3: Union bound and asymptotical approximation of soft correction codeword error rate for given values of E_b/N_0 using simulation

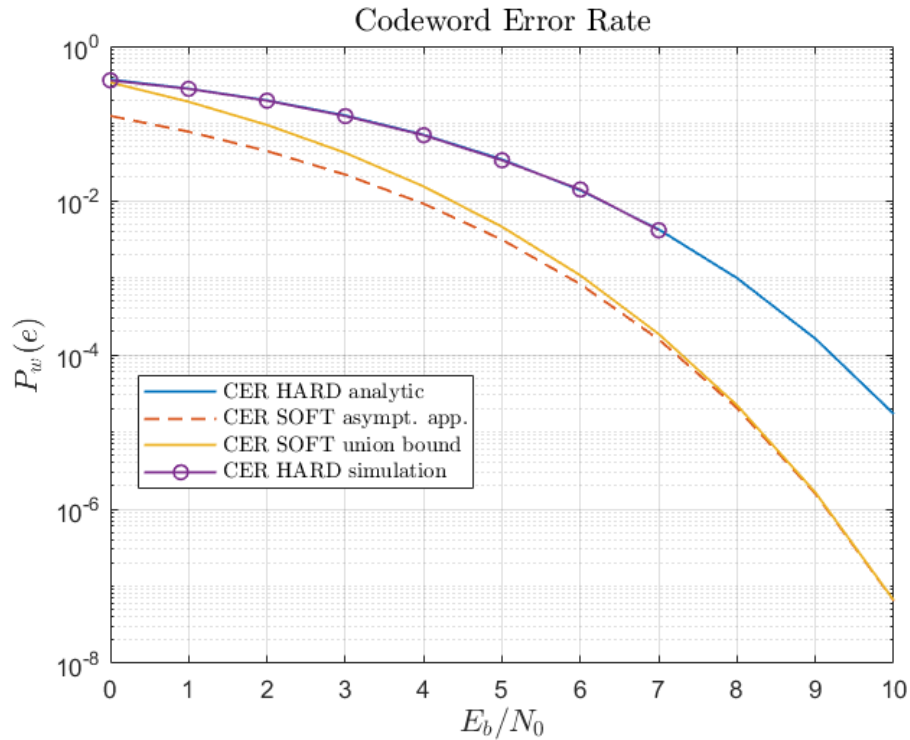


Figure 1-3: Union bound and asymptotical approximation of soft correction codeword error rate

Now for the simulation part, we do the same procedure as hard correction until we obtain the noisy vector r . Instead of looking at the sign of r and deciding for bits, we compute the Euclidean distance between r and all existing c 's and we find the c' with the minimum Euclidean distance and declare it as corrected codeword. We check if this codeword is equal to the transmitted or not. If not, we add to the number of errors. The results of simulation part are given in Table 1-4 and the final plot is shown in Figure 1-4.

$\frac{E_b}{N_0} [dB]$	Codeword error rate
0	1.719×10^{-1}
1	1.170×10^{-1}
2	6.202×10^{-2}
3	3.166×10^{-2}
4	1.322×10^{-2}
5	4.061×10^{-3}
6	9.705×10^{-4}
7	1.934×10^{-4}

Table 1-4: Codeword error rate of error correction soft for given values of E_b/N_0 using simulation

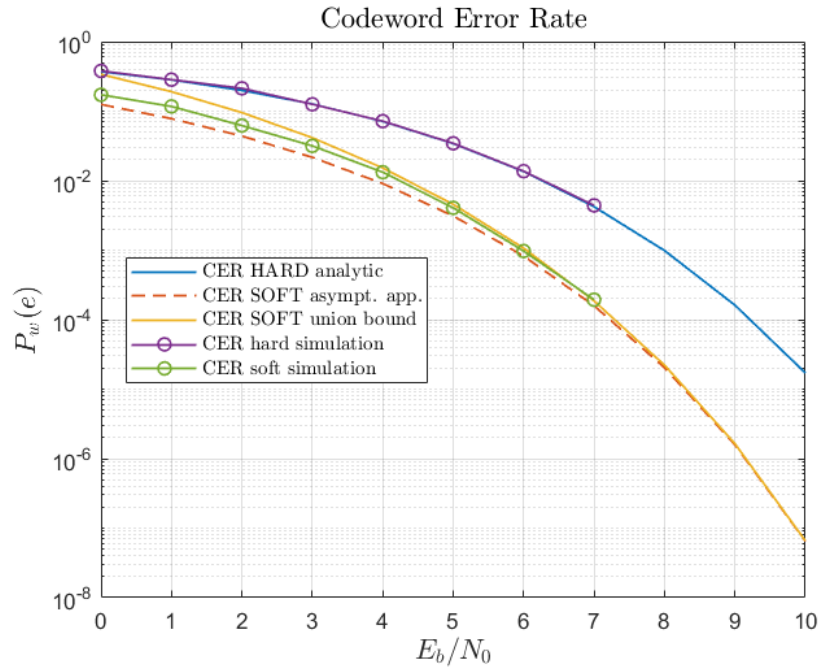


Figure 1-4: Hard correction codeword error rate using formula and simulation and soft correction codeword union bound, asymptotical approximation, and simulation

From the figures and tables, we observe that for hard correction, simulation and analytical formula results completely match each other. Also we see that true value of soft correction which is obtained by simulation, has a value between union bound and asymptotical approximation. We can see that for high $\frac{E_b}{N_0}$ these three values match each other.

An important result we get from the figures is that for a given $\frac{E_b}{N_0}$, soft correction performs better than hard correction. This is because in the hard correction we only use the sign of the received codeword but in the soft correction we use both sign and amplitude of the received codeword and which enhances the performance and reduces the codeword error rate. Also we can observe that as $\frac{E_b}{N_0}$ increases, the difference between performance of soft correction and hard correction also increases.

For additional part, first, we generate an LUT with 2^r rows and then we assign errors with weight 1 to their corresponding syndrome. After this, we had to complete table with errors with weight 2 (and if needed, weight 3 or more). After filling the table which is given in Table 1-5, we follow the same procedure for hard correction and the final results are shown in Figure 1-5.

Syndrome	Error Vector
0000	00000000
0001	00000001
0010	00000010
0011	01001000
0100	00000100
0101	00101000
0110	10000000
0111	10000001
1000	00001000
1001	00001001
1010	00001010
1011	01000000
1100	00010010
1101	00100000
1110	00010000
1111	01000100

Table 1-5: Full LUT table

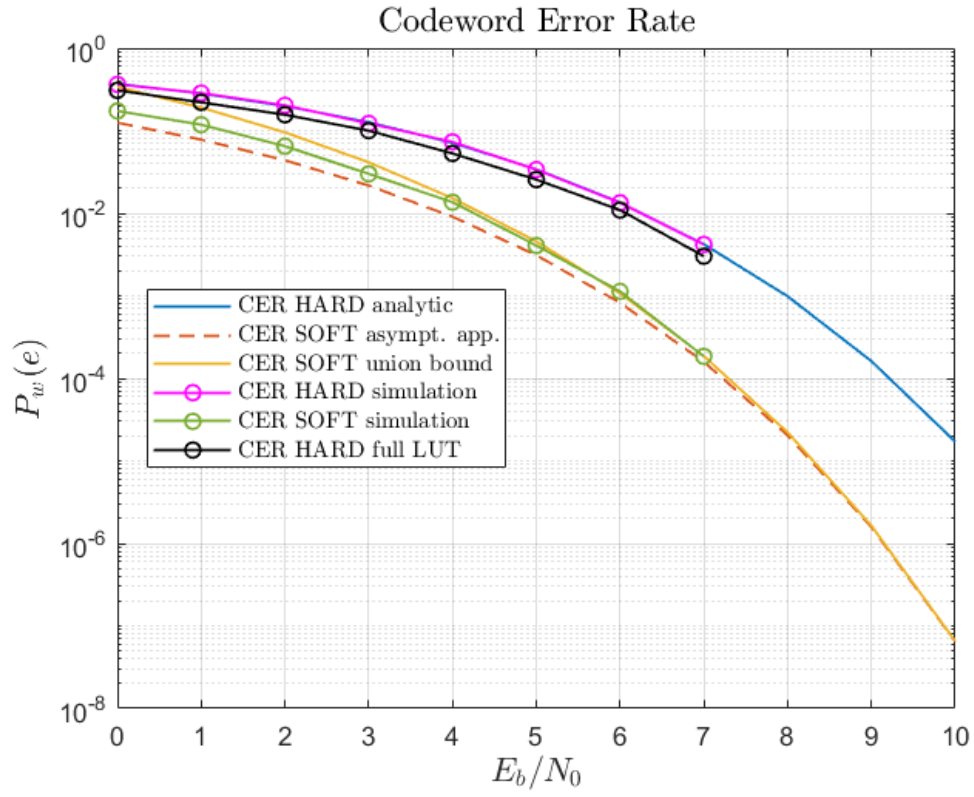


Figure 1-5: Hard correction codeword error rate using formula and simulation and soft correction codeword union bound, asymptotical approximation, and simulation and hard correction using full LUT

Exercise 2

First, we produce a 127-symbol of output called $v(n)$. Then we map these symbols to bipolar $v'(n)$. Then we compute N_0 which is the number of zeros in this sequence, N_1 which is the number of ones in this sequence, N_T which is the number of transitions in this sequence, and N_{NT} which is the number of no transitions.

A transition happens when two consecutive bits are not equal. For an ideal m-sequence, $N_0 = N_1 = \frac{N}{2}$, $N_T = N_{NT} = \frac{N}{2}$. We run the code and get the result which is shown in Table 1-2.

	Ideal m-sequence	Real m-sequence
N_0	64	63
N_1	64	64
N_T	64	64
N_{NT}	64	63

Table 2-1: Comparing the properties of m-sequences

The difference between the real properties and ideal properties happens because we do not have all zero state. The plot for bipolar sequence is shown in Figure 2-1.

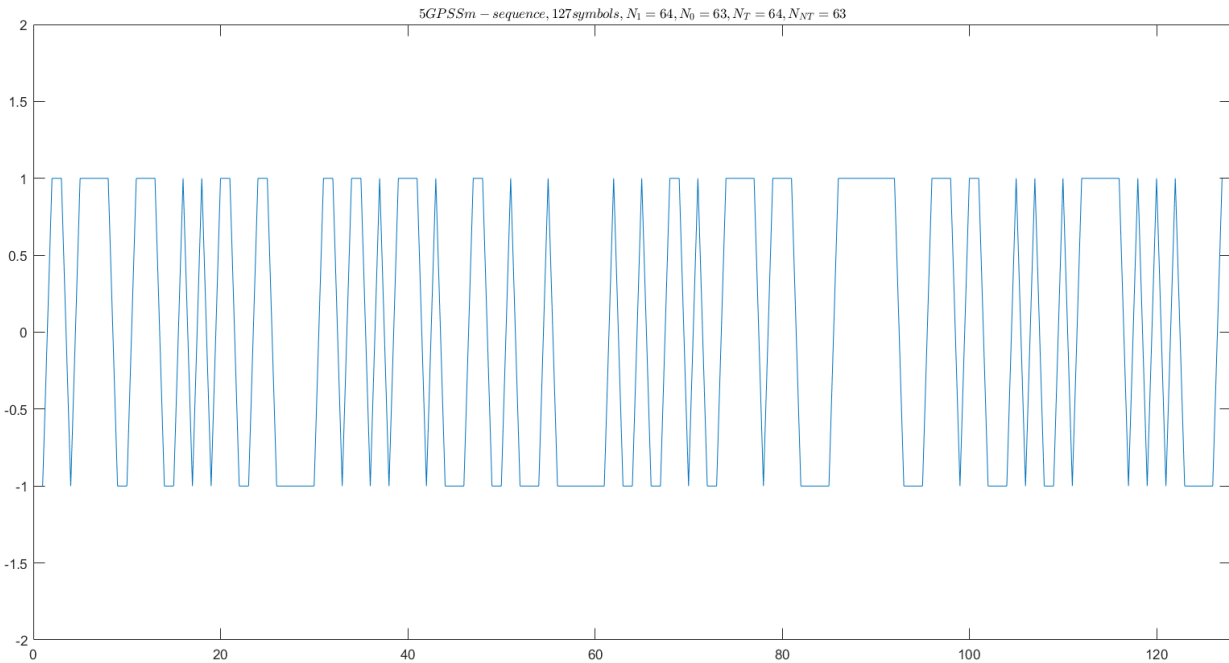


Figure 2-1: Bipolar representation of the sequence generated by LFSR

Now we want to compute the number of runs. A run of length i is a sequence of i consecutive equal bits. For an ideal random binary sequence, we have this equation:

$$\text{Number of runs of length } i = \frac{N_R}{2^i}, \quad N_R = \text{Total number of runs}$$

Also, we expect that if we have $\frac{N_R}{2}$ runs of length 1, half of them are runs of zeros and half of them are runs of ones. Now for this LFSR we compute the number of runs and the results are given in Table 2-2 and the runs with their starting position are shown in Figure 2-2. The result tells us that $N_R = 64$

Run length	Real number of runs	Number of zero runs	Number of one runs
1	32	16	16
2	16	8	8
3	8	4	4
4	4	2	2
5	2	1	1
6	1	1	0
7	1	0	1

Table 2-2: Number of runs for the given LFSR

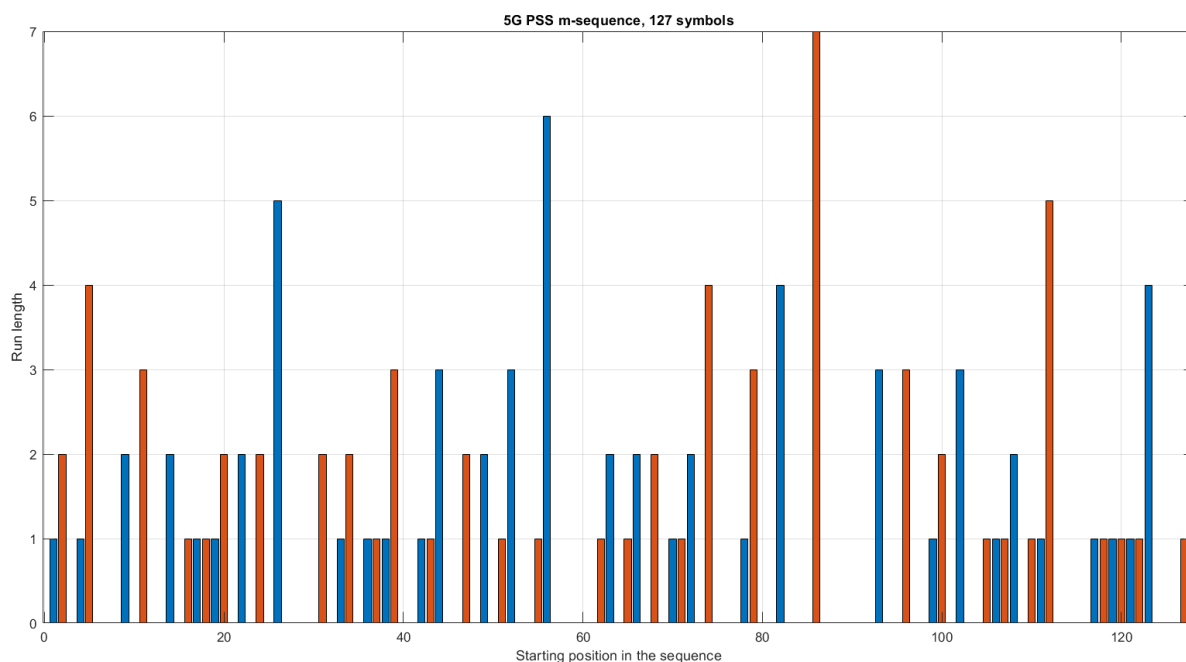


Figure 2-2: Number of runs shown in blue (zero runs) and orange (one runs)

We observe that properties of this binary sequence approximately satisfy the properties for an ideal random binary sequence.

For autocorrelation we use the command given in assignment (which proof is in question 3) and we get the Figure 2-3.

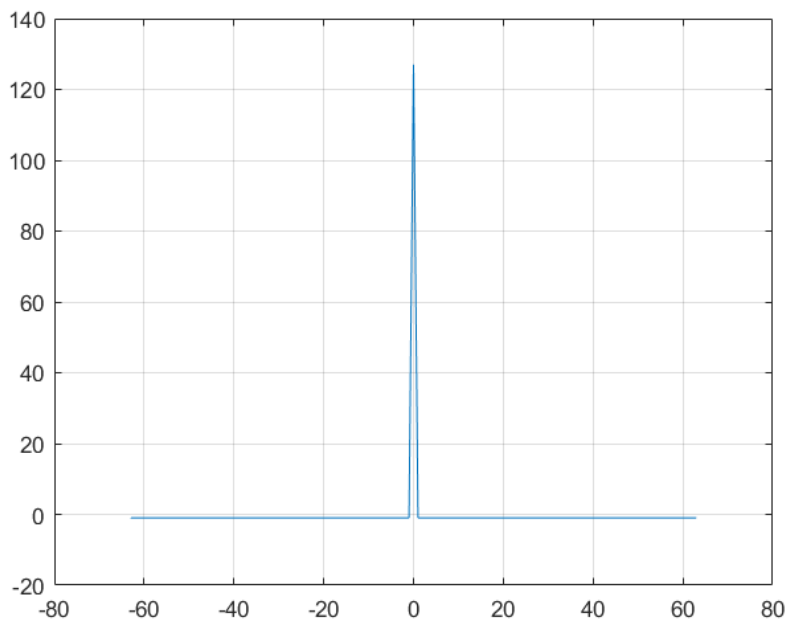


Figure 2-3: Autocorrelation for the given binary sequence

We know that for an ideal random binary sequence, Autocorrelation function is:

$$R(\tau) = \begin{cases} N & , \quad \tau = 0 \\ 0 & , \quad \tau \neq 0 \end{cases}$$

For this random binary sequence, we have:

$$R(\tau) = \begin{cases} N & , \quad \tau = 0 \\ -1 & , \quad \tau \neq 0 \end{cases}$$

We show that this property is very close to ideal random binary sequence.

For the next part, at first, we define truncated m-sequence and MPSL and then we try to compute the MPSL. Truncated m-sequences are obtained by taking an entire m-sequence and deleting the last x bits. We define Maximum Peak Side Lobe (MPSL) as:

$MPSL \equiv \max |R(\tau)|, \tau \neq 0$. For an ideal random binary sequence $MPSL=0$ and for a non truncated m-sequence $MPSL=1$.

At first, we drop the last 10 bits and compute the MPSL, then we repeat this for five different starting seed and the result is given in Table 2-3.

Starting seed	MPSL
1110110 (main)	13
1011101	17
1001100	13
1101100	21
1010101	13
0001000	17

Table 2-3: MPSL values for different starting seeds for the given LFSR

As we observe, the value of MPSL has increased significantly in comparison with the non-truncated m-sequence. For many different seeds this procedure was repeated and the minimum MPSL was 13 and the maximum was 21.

Exercise 3

For this exercise, as the assignment has expressed, we fix the marker length and generate a random binary pattern and map it to a bipolar sequence. Now we fix SNR to 10 dB and we convert it to its real value and by using the real value, we compute the sigma for creating the noise pattern by using the formula $SNR = \frac{E_s}{\sigma^2}$. We define $r_0 = n$ and $r_1 = p' + n$ as the received signal under H_0 and H_1 , respectively.

Now we compute statistical tests T_0 and T_1 which are defined generally as:

$T(r) = \sum_{i=1}^N r_i p'_i$ which is the soft correlation between received vector r and the bipolar pattern p'_i . We plot the T_0 and T_1 which are shown in Figure 3-1 and Figure 3-2, respectively.

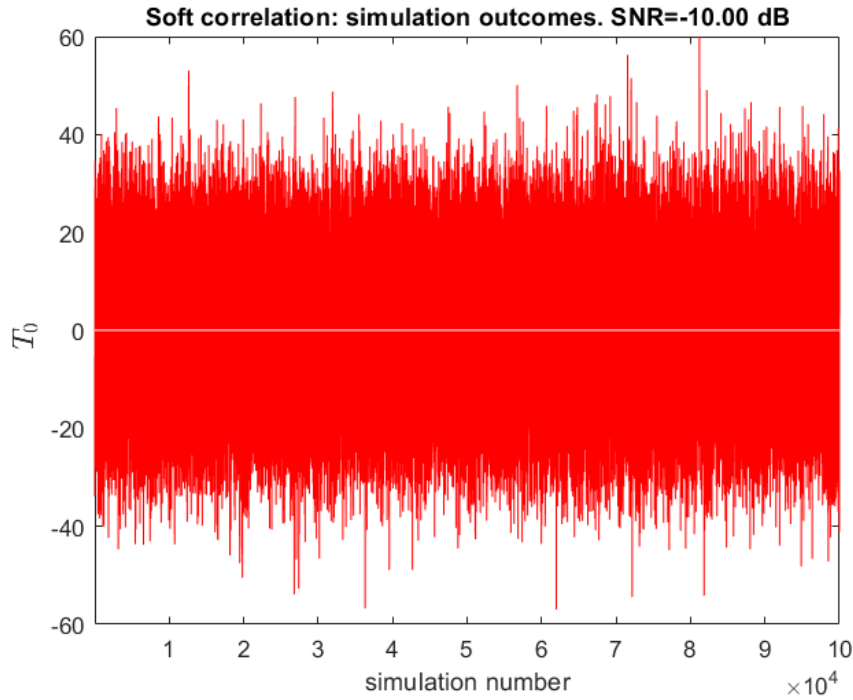


Figure 3-1: T_0 outcome and its mean for $SNR = -10 \text{ dB}$

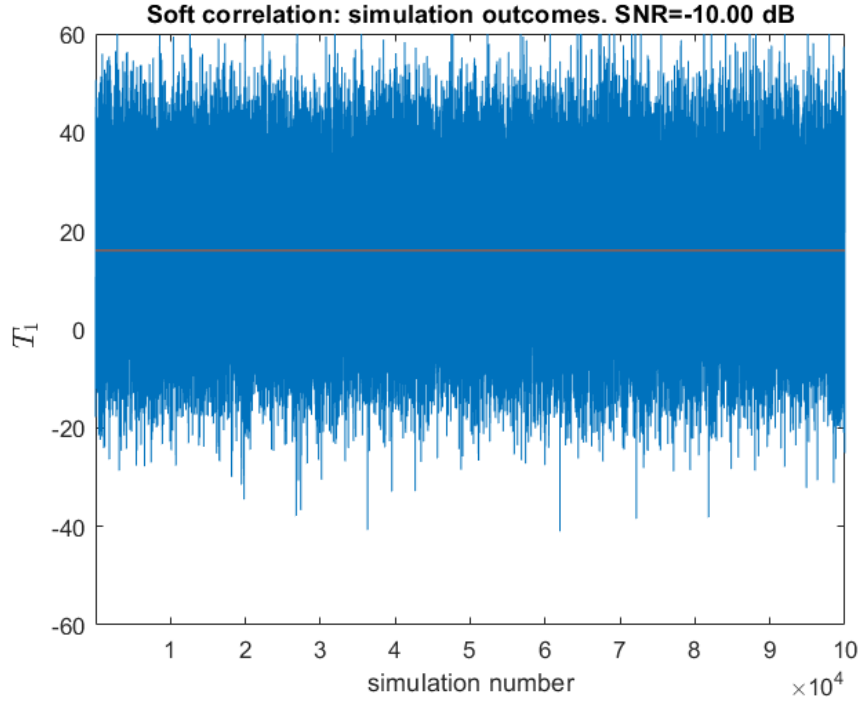


Figure 3-2: T_1 outcome and its mean for $SNR = -10 \text{ dB}$

As we expected, mean value of T_0 is approximately equal to zero and mean value of T_1 is approximately equal to $N = 16$.

Now we use histogram command to plot the PDFs of T_0 and T_1 . The plot is shown in Figure 3-3.

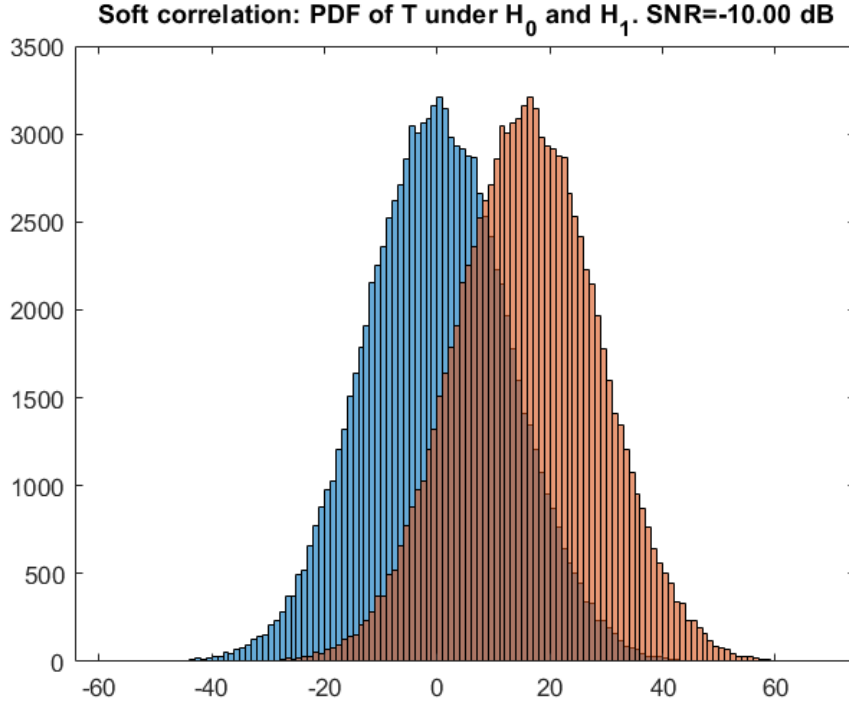


Figure 3-3: PDF of T_0 (blue curve) and T_1 (orange curve) for $SNR = -10 \text{ dB}$

We can see that two PDFs have a significant overlap with each other which is not a good property because it leads to having a high value of P_{fa} and P_{md} . We have a false alarm when we decide H_1 and H_0 is true and we have a missed detection when we decide H_0 and H_1 is true.

For the next part, at first we have to define the threshold. Threshold is the value on which we decide H_0 and H_1 . We change the threshold from the minimum of T_0 to the maximum of T_1 . We count the number of false alarm and missed detection events and divide the number of simulations by them. We have a false alarm if we are under the H_0 but we are above the threshold and we have a missed detection if we are under the H_1 but we are below the threshold.

We want to have a low value for both P_{fa} and P_{md} . But when we want to decrease P_{md} we decrease the threshold but P_{fa} increases and when we want to decrease P_{fa} we increase the threshold but P_{md} also increases. So, we will have a trade off between these two parameters and the two curves intersect at $threshold = 6$. We will observe this in Figure 3-4.

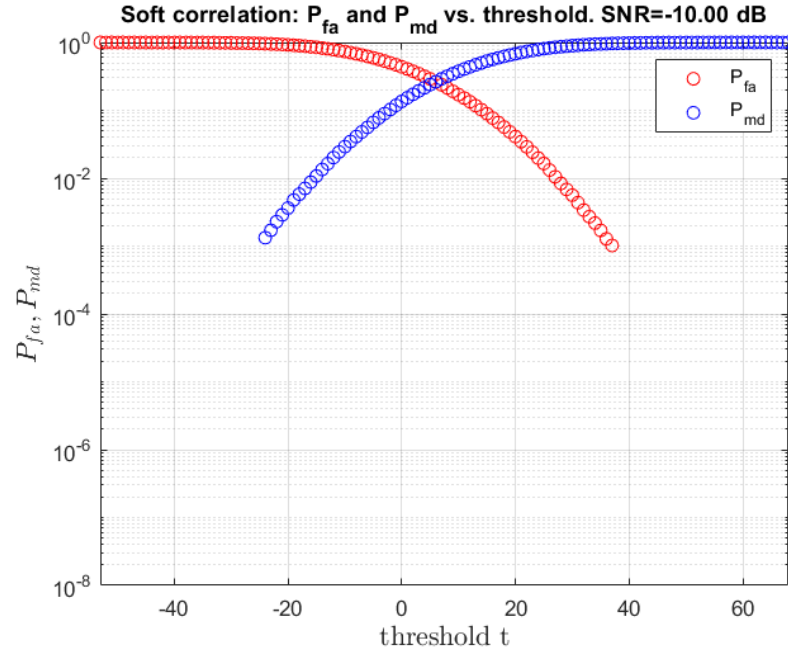


Figure 3-4: Plotting the probability of false alarm and missed detection with respect to threshold for $SNR = -10 \text{ dB}$

Now we plot ROC curve which is probability of detection against probability of false alarm.

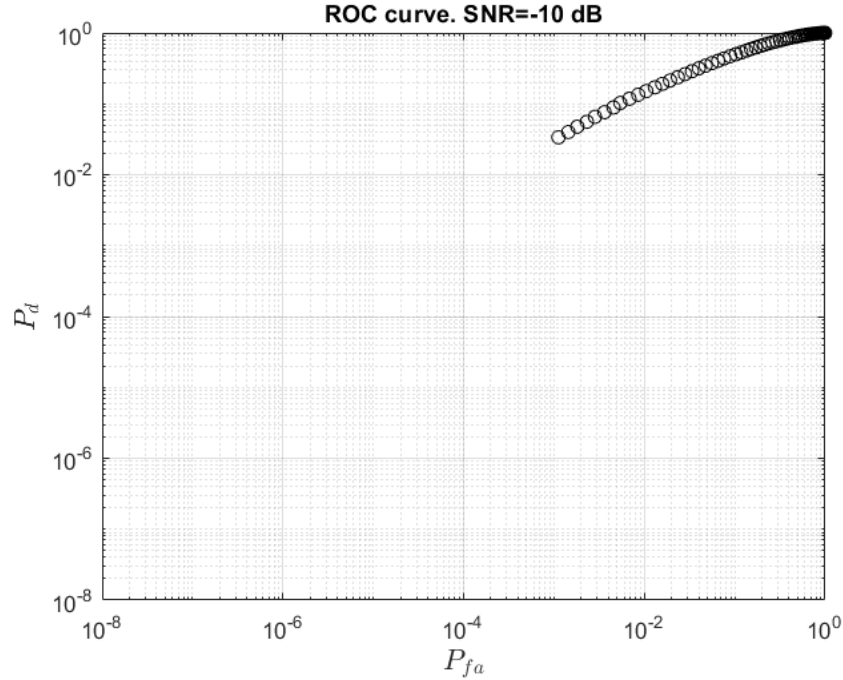


Figure 3-5: ROC curve for $SNR = -10 \text{ dB}$

This curve matches our previous expression. If we want to have a high P_d then we will have a high P_{fa} and if we want to decrease the P_{fa} we will decrease the P_d .

Now we fix the $N=16$ and repeat this procedure for $SNR = 0, 10$ dB.

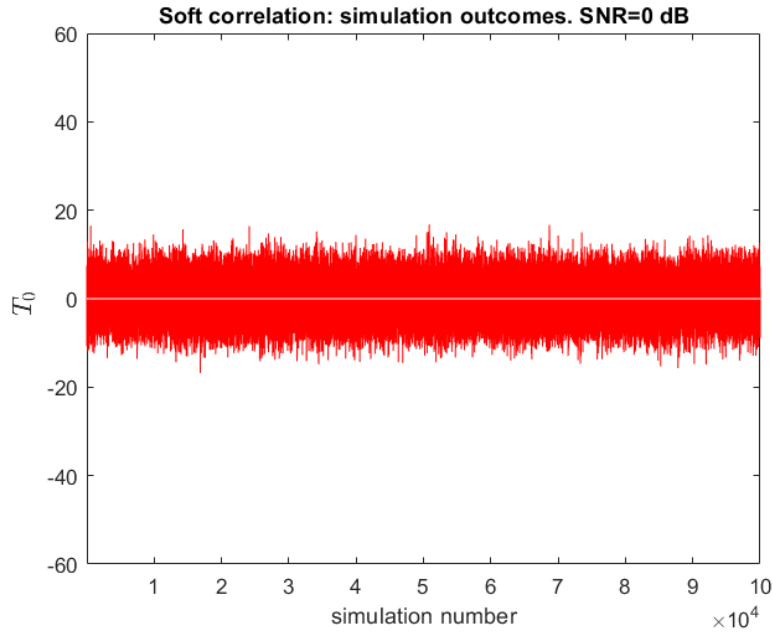


Figure 3-6: T_0 outcome and its mean for $SNR = 0$ dB

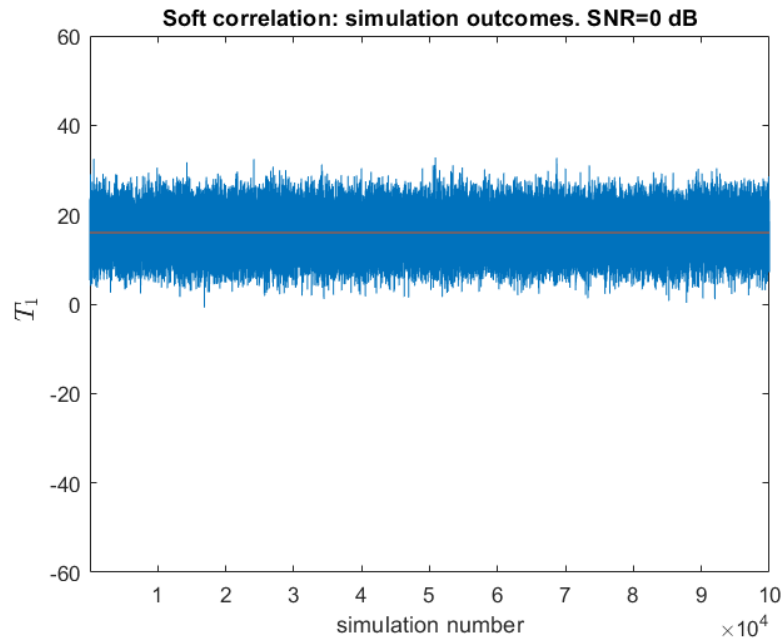


Figure 3-7: T_1 outcome and its mean for $SNR = 0$ dB

We can observe that the fluctuations and the variations of the noises have been decreased with increasing SNR but mean value remains at 0 , 16.

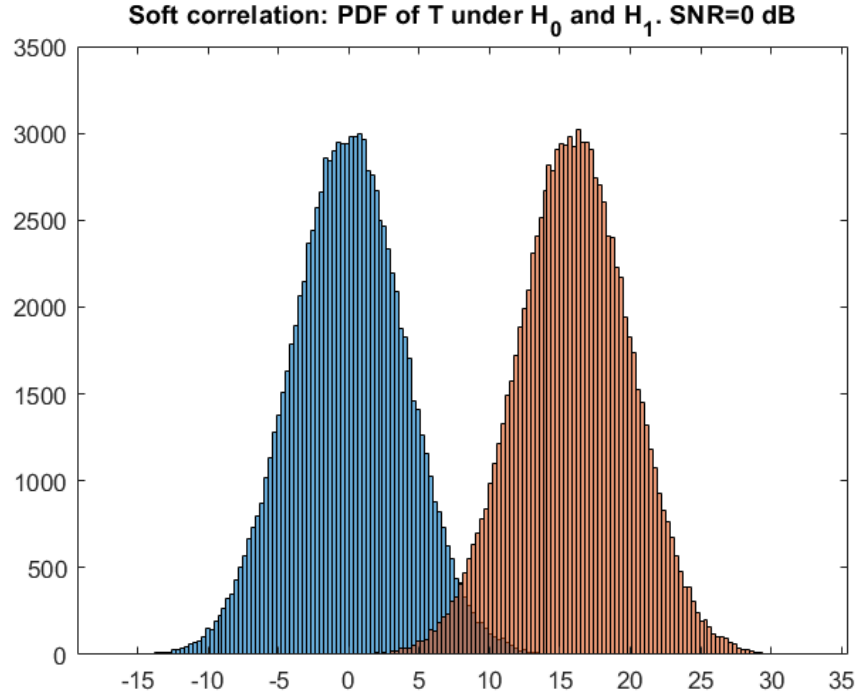


Figure 3-8: PDF of T_0 (blue curve) and T_1 (orange curve) for $SNR = 0$ dB

We can clearly see that the overlap between the two curves has been decreased and therefore probability of false alarm and missed detection will increase.

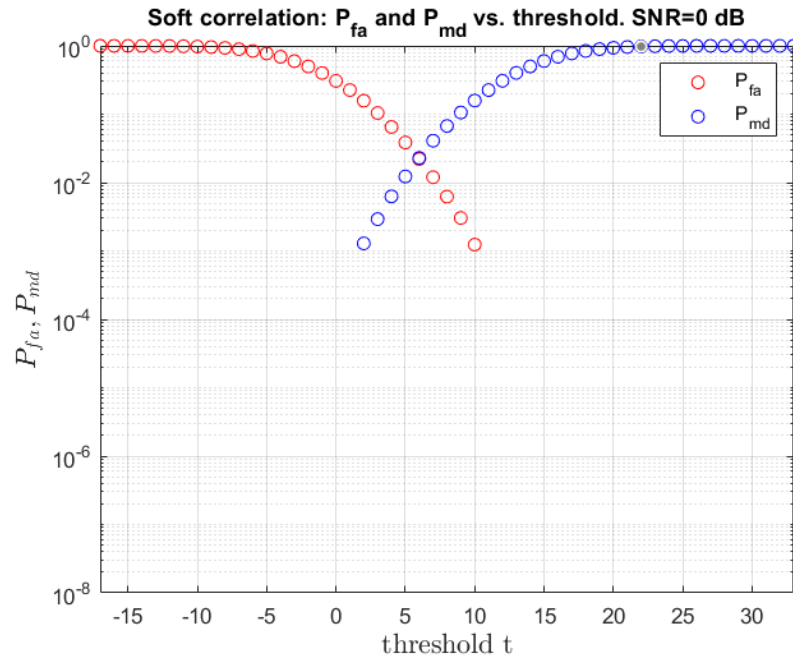


Figure 3-9: Plotting the probability of false alarm and missed detection with respect to threshold for $SNR = 0$ dB

We can understand that the slope of decreasing for both curves has increased and they have a lower probability for a certain threshold (in comparison with $SNR = -10dB$) but they intersect each other at $threshold = 6dB$ which is equal to the same figure for $SNR = -10dB$.

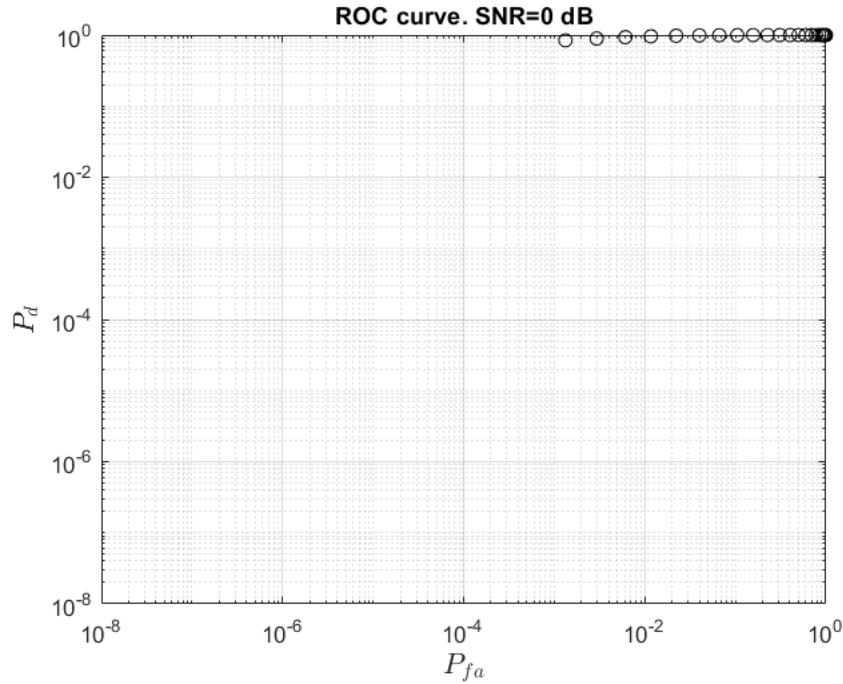


Figure 3-10: ROC curve for $SNR = 0 dB$

We can understand that for a higher SNR, we can decrease the P_{fa} and the P_d approximately stays constant and almost equal to 1 which proves that by having a high SNR probability of false alarm and missed detection will decrease.

In the following pages, we have figures for $SNR = 10dB$.

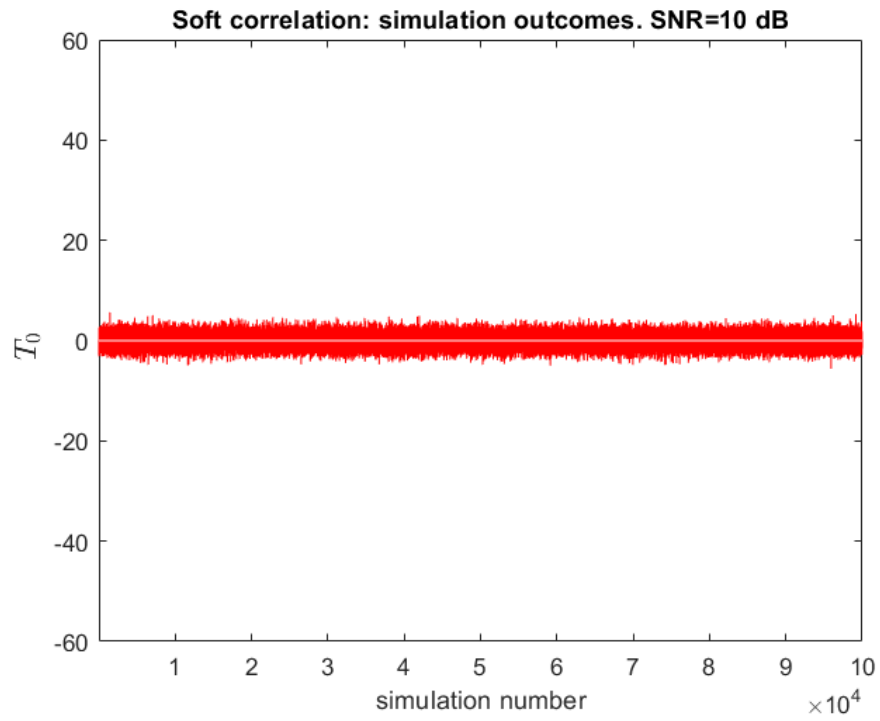


Figure 3-11: T_0 outcome and its mean for $SNR = 10 \text{ dB}$

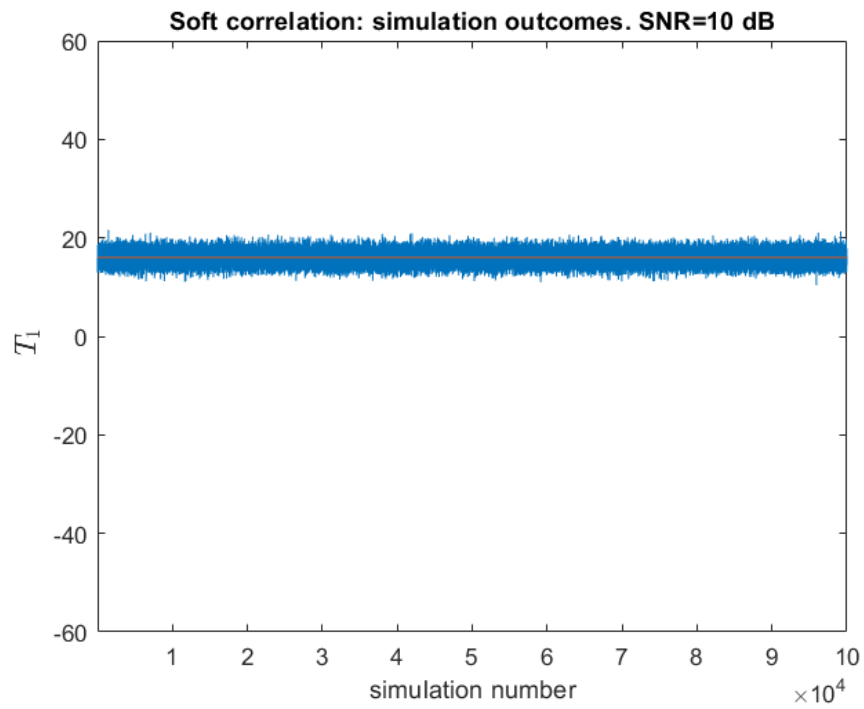


Figure 3-12: T_1 outcome and its mean for $SNR = 10 \text{ dB}$

We can observe that the fluctuations for noises have decreased more and the outcomes are more concentrated and mean value is still equal to 0, 16.

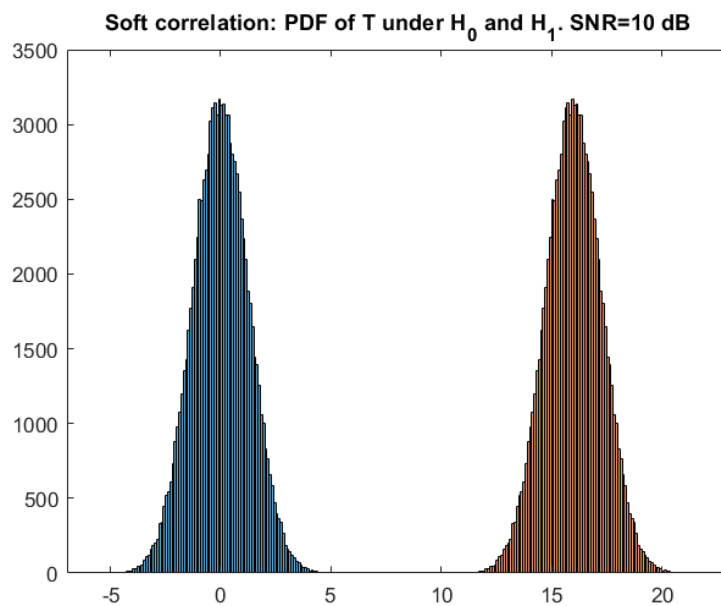


Figure 3-13: PDF of T_0 (blue curve) and T_1 (orange curve) for $SNR = 10 \text{ dB}$

We can see that by increasing SNR to 10dB, the two histograms are separated and therefore probability of false alarm for a proper threshold which is between the two histograms are negligible.

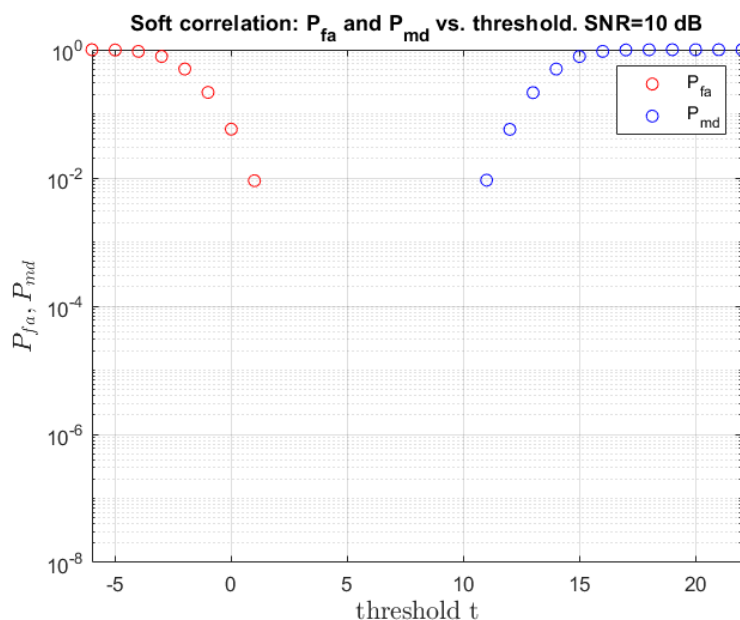


Figure 3-14: Plotting the probability of false alarm and missed detection with respect to threshold for $SNR = 10 \text{ dB}$

We can observe that for a threshold between 1 and 11 dB the probability of false alarm and missed detection are less than 0.01.

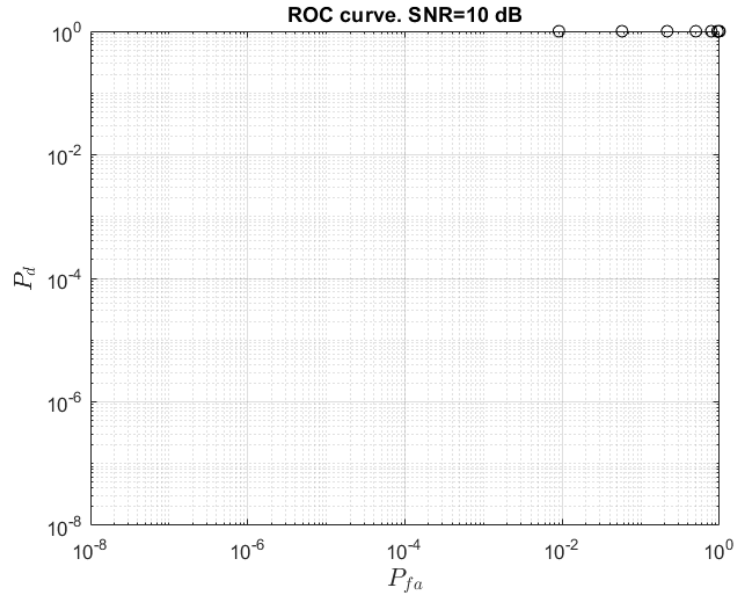


Figure 3-15: ROC curve for $SNR = 10 \text{ dB}$

For the next part, we fix $SNR=10 \text{ dB}$ and change the N from 16 to $N = 8$ and $N = 32$.

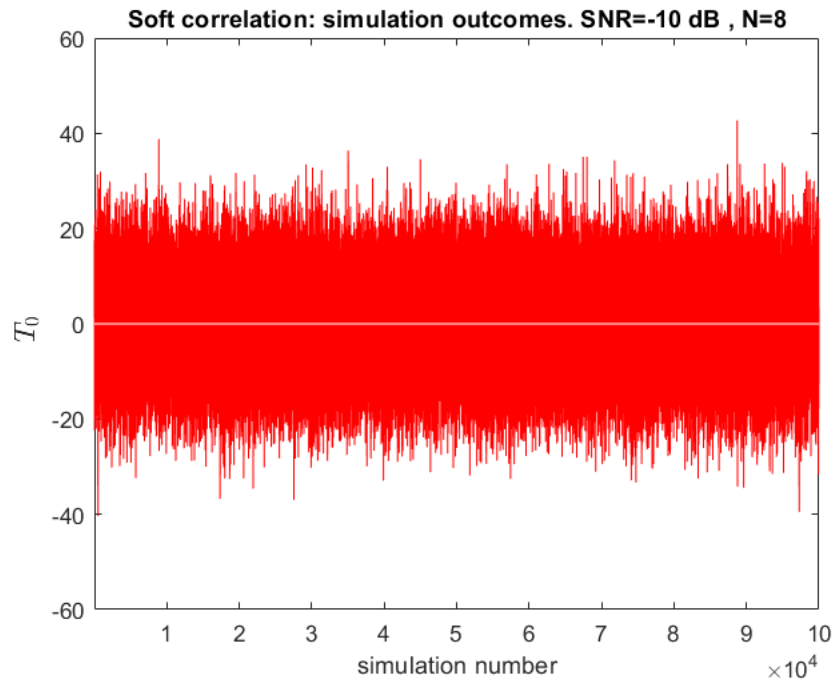


Figure 3-16: T_0 outcome and its mean for $SNR = -10 \text{ dB}$, $N = 8$

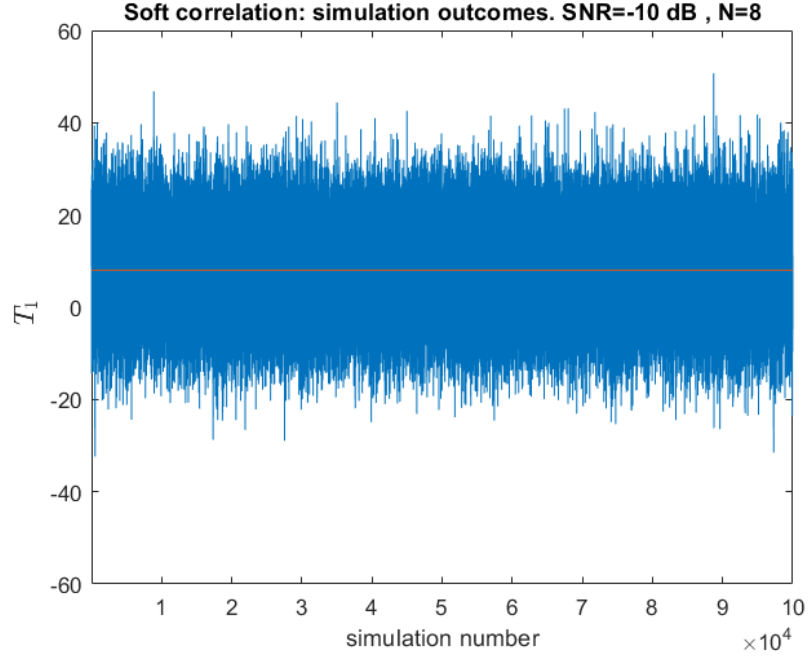


Figure 3-17: T_1 outcome and its mean for $SNR = -10\text{ dB}$, $N = 8$

We can observe that by decreasing the N , amplitude of noises has been decreased but mean value remains constant.

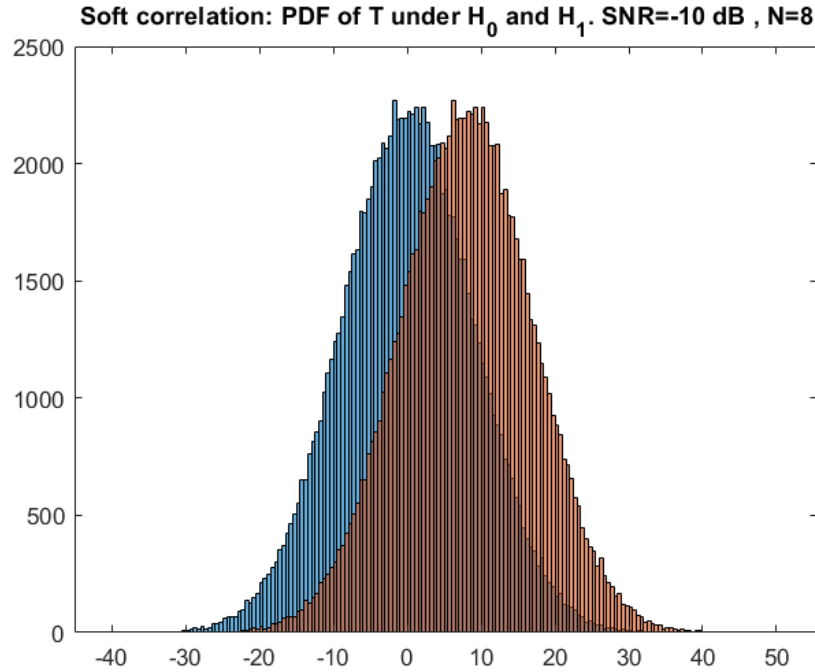


Figure 3-18: PDF of T_0 (blue curve) and T_1 (orange curve) for $SNR = -10\text{ dB}$, $N = 8$

It is understood that the overlap of the two PDFs has increased with decreasing the N .

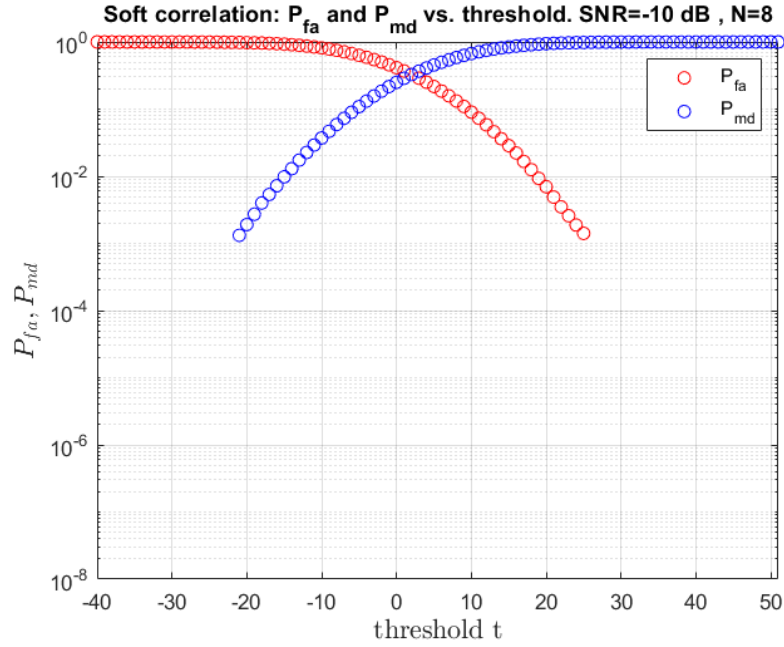


Figure 3-19: Plotting the probability of false alarm and missed detection with respect to threshold for $SNR = -10\text{ dB}$, $N = 8$

We can see that two curves intersect at a lower threshold ($threshold = 2$) and also, the rate of decreasing the curves has decreased.

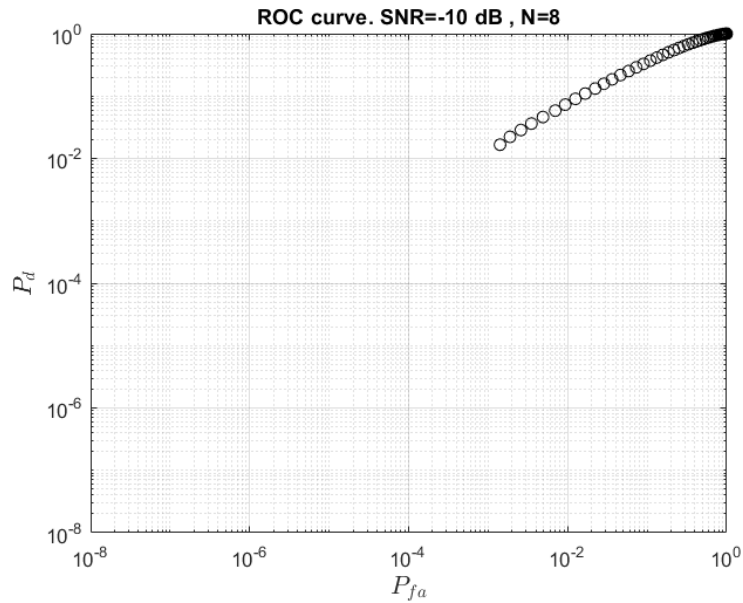


Figure 3-20: ROC curve for $SNR = -10\text{ dB}$, $N = 8$

It is noticeable that slope for ROC has been decreased or in another way, for a certain probability of false alarm, probability of detection has decreased by decreasing N .

The results for $SNR = -10 \text{ dB}$, $N = 32$ are given in the following.

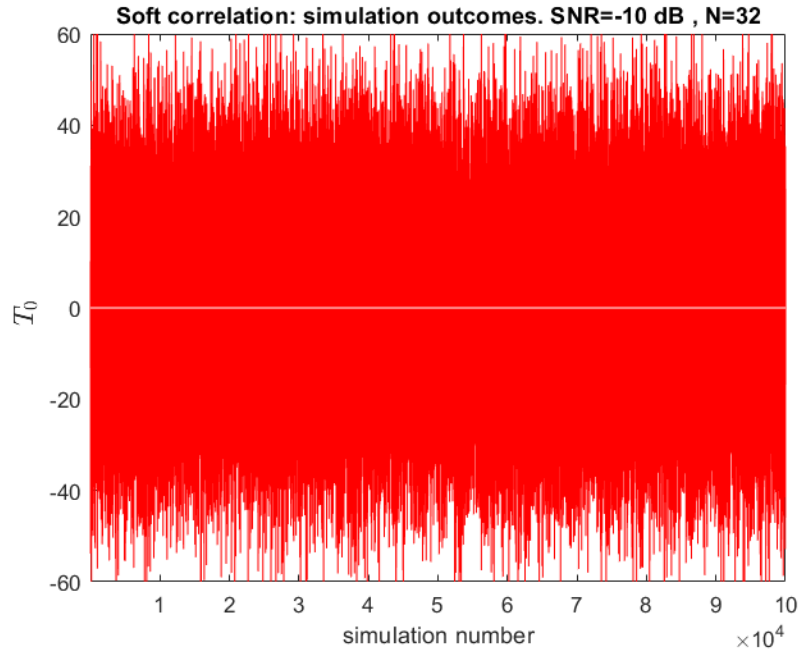


Figure 3-21: T_0 outcome and its mean for $SNR = -10 \text{ dB}$, $N = 32$

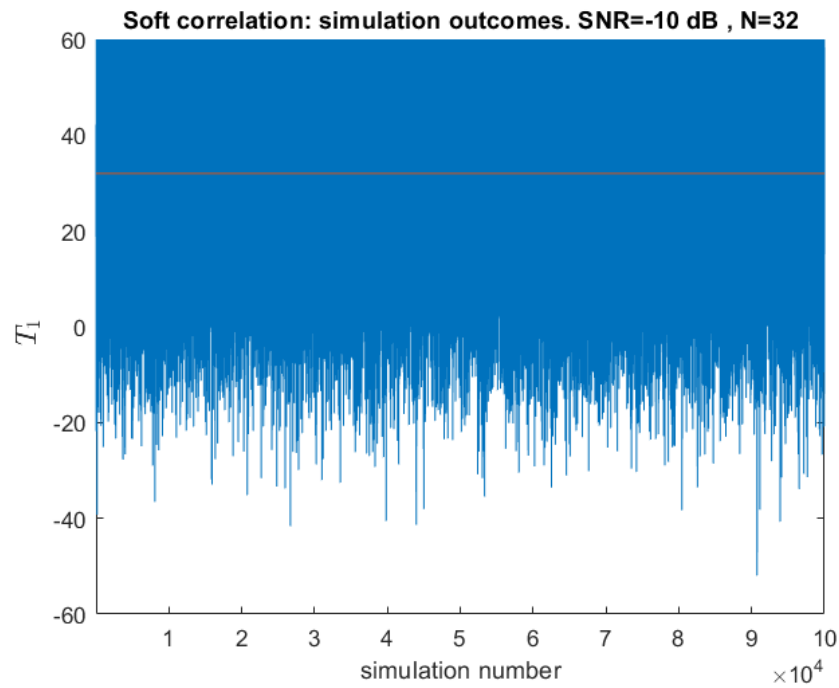


Figure 3-22: T_1 outcome and its mean for $SNR = -10 \text{ dB}$, $N = 32$

We can see that the fluctuations has been increased and the mean values are 0 and 32.

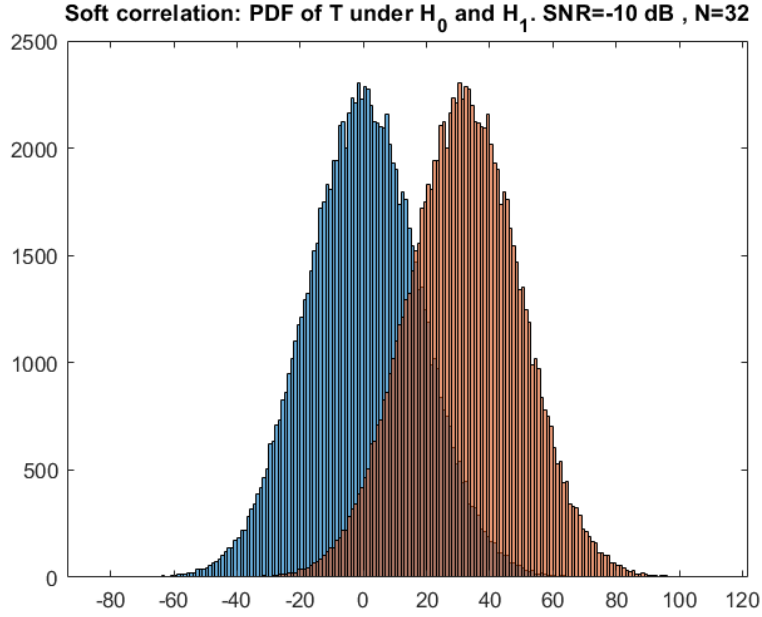


Figure 3-23: PDF of T_0 (blue curve) and T_1 (orange curve) for $SNR = -10 \text{ dB}$, $N = 32$

We can understand that by increasing the N , the overlap between two PDFs decreases but this decrease is less than the decrease caused by increasing SNR.

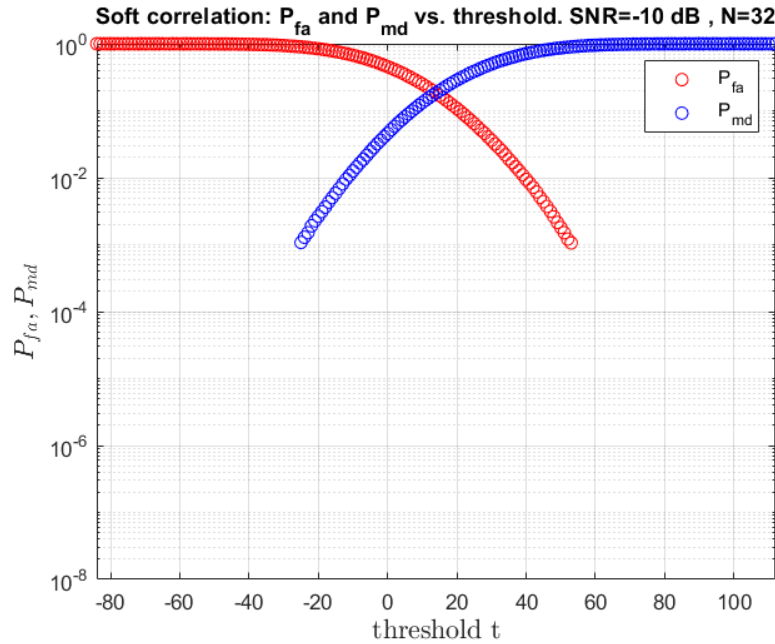


Figure 3-24: Plotting the probability of false alarm and missed detection with respect to threshold for $SNR = -10 \text{ dB}$, $N = 32$

We can observe that the threshold in which the two curves intersect has increased to 14 and, they reach $p = 1$ faster than previous figures.

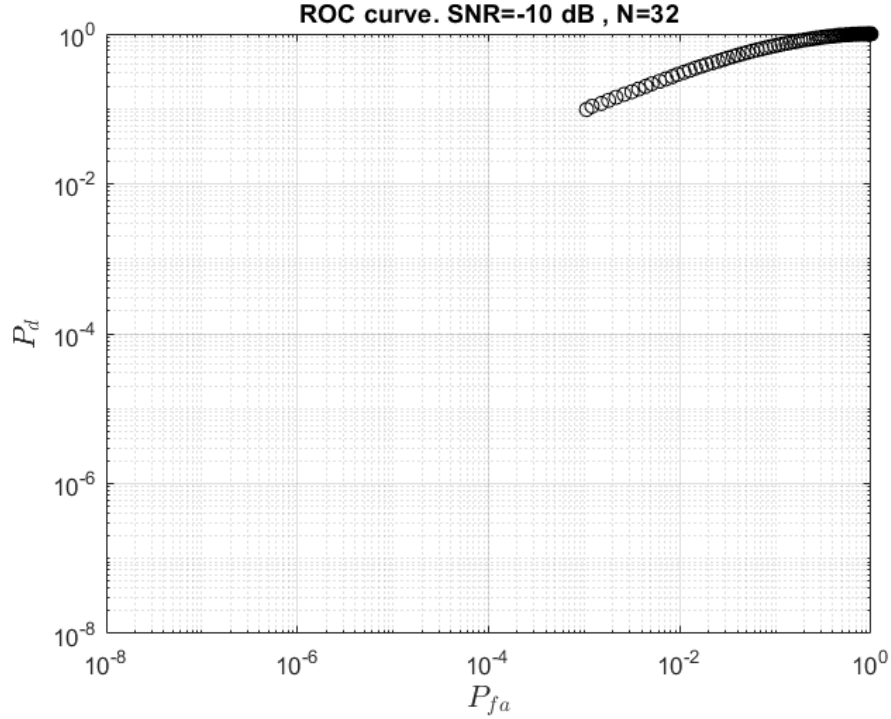


Figure 3-25: ROC curve for $SNR = -10 \text{ dB}$, $N = 32$

We can see that slope for ROC has increased by increasing N , but this increase is less than the increase for increasing SNR.

We summarize the results in a table.

	Increasing SNR	Increasing N
Mean values of simulation outcomes	Constant	Constant
Amplitude of noises in simulation outcomes	Decreases	Increases
Two PDFs overlap	Significantly increases	Slightly increases
P_{fa} and P_{md} intersection	Constant	Increases
Slope of ROC	Significantly increases	Slightly increases

Table 3-1: Summarizing the obtained results

To sum it up, by increasing SNR, two PDFs will have less overlap and therefore probability of missed detection and false alarm decreases. Also, for a fixed SNR, changing the threshold will change the probabilities but they have a trade off and we cannot reduce both by regulating the threshold.

For the analytical part, we compute the probability of false alarm and missed detection using erfc function in MATLAB and we plot the curves which are shown in Figure 3-26 and Figure 3-27.

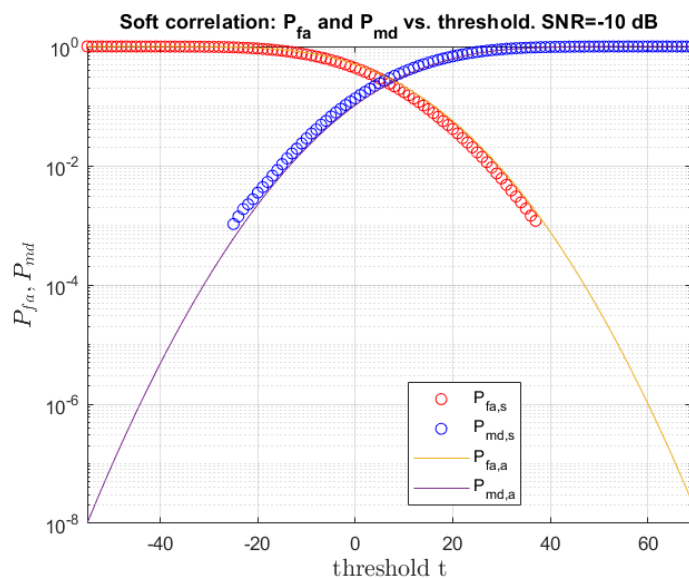


Figure 3-26: Probability of false alarm and missed detection with threshold for simulation and analytics

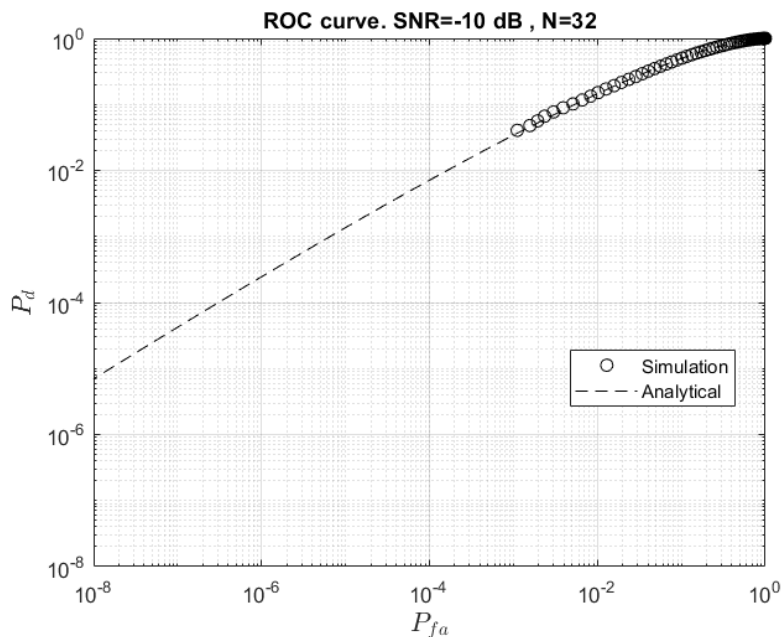


Figure 3-27: ROC curve for simulation and analytics

We can see that simulation results and analytical results match each other.

For the last part, we fix the probability of false alarm to 0.01 and 0.0001 and change the SNR from -10 to $+8$ and see what happens for probability of missed detection. The result is shown in Figure 3-28.

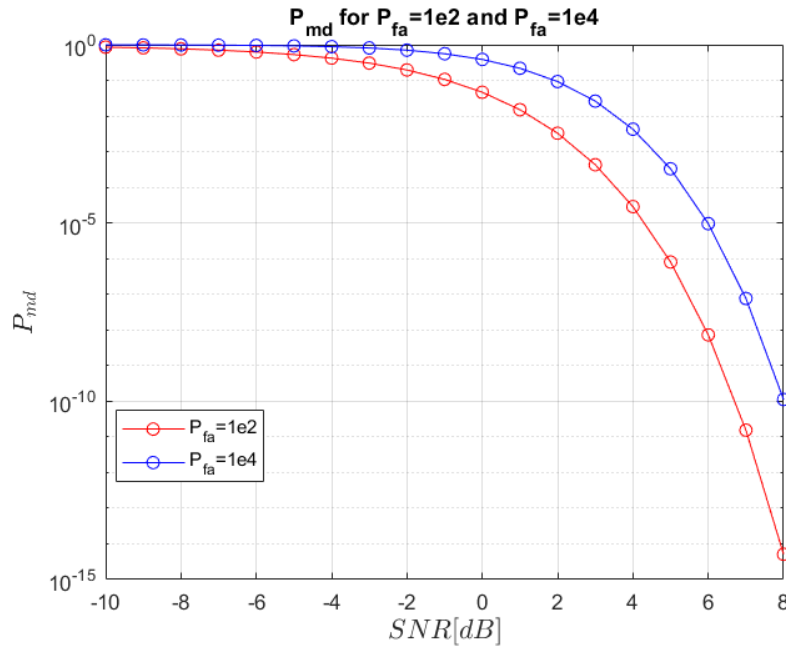


Figure 3-28: probability of missed detection for fixed false alarm rate with respect to different SNRs

We can observe that for a fixed SNR, the probability of missed detection for the $P_{fa} = 0.0001$ is higher than the probability of missed detection for the $P_{fa} = 0.01$. Also, we can see that for low SNR, there is not much difference between two curves but for high SNR this difference becomes significant.

Note: For the analytical part, I inspired from the code written by Professor Galleani in Signal Processing course which computes the Q function. I used his notation for writing the code using erfc function.

Question 1

-Extension: If the code C has an even minimum distance d , the minimum distance does not change by extending because we add zero bits to all codewords. Instead, if it has an odd minimum distance, by increasing it will raise by 1 because we add zero bits to the even codewords and 1 to the odd codewords.

Consider $C1$ below which has a minimum distance 3. We create $C2$ which is the extension of $C1$ and see that minimum distance is increased to 4.

$C1$:

```
000000
100110
010101
110011
001011
101101
011110
111000
```

$C2$:

```
0000000
1001101
0101011
1100110
0010111
1011010
0111100
1110001
```

But if the code has an even minimum distance, the extended parity bit will be zero and nothing changes therefore minimum distance remains constant.

-Puncturing: If the code C has a minimum distance d , the minimum distance will decrease by 1 by puncturing because minimum distance is related to the parity bits and therefore by removing a parity bit minimum distance is reduced.

Consider $C3$ and $C5$ which are two codes with an even minimum distance. We show that their minimum distance will decrease by 1 by puncturing.

$C3$:

0000
0011
0101
0110
1001
1010
1100
1111

$C4$:

000
001
010
011
100
101
110
111

We can see that minimum distance has decreased to 3.

$C5$:

00000000
10000111
01001011
11001100
00101101
10101010
01100110
11100001
00011110
10011001
01010101
11010010
00110011
10110100
01111000
11111111

C_6 :

```
0000000
1000011
0100101
1100110
0010110
1010101
0110011
1110000
0001111
1001100
0101010
1101001
0011001
1011010
0111100
1111111
```

We can observe that minimum distance has decreased again and it independent from the parity bit which we delete, and, in all case, it will decrease by 1. In next page we introduce C_7 and C_9 which have an odd minimum distance and see that puncturing has what effect on their minimum distance.

C_7 :

```
000000
100110
010101
110011
001011
101101
011110
111000
```

C_8 :

```
00000
10011
01010
11001
00101
10110
01111
11100
```

The minimum distance has decreased by puncturing.

C_9 :

```
00000000
10000110
01001011
11001101
00101101
10101011
01100110
11100000
00011110
10011000
01010101
11010011
00110011
10110101
01111000
11111110
```

C_{10} :

```
00000000
1000010
0100101
1100111
0010111
1010101
0110010
1110000
0001110
1001100
0101011
1101001
0011001
1011011
0111100
1111110
```

We can observe that apart from having an odd or even minimum distance, minimum distance will decrease by doing puncturing because we remove a parity bit.

-Shortening: If the code C has a minimum distance d , the minimum distance will remain constant by shortening. Consider C_{11} and C_{13} which have an even minimum distance and C_{15} which has an odd minimum distance.

C_{11} :

0000
0011
0101
0110
1001
1010
1100
1111

C_{12} :

000
011
101
110

We see that minimum distance for C_{12} is still 2.

C_{13} :

00000000
10000111
01001011
11001100
00101101
10101010
01100110
11100001
00011110
10011001
01010101
11010010
00110011
10110100
01111000
11111111

C_{14} :

0000000
1001011
0101101
1100110
0011110
1010101
0110011
1111000

We see that minimum distance for C_{14} is still 4.

C_{15} :

000000
100110
010101
110011
001011
101101
011110
111000

C_{16} :

00000
10101
01011
11110

We can see that by shortening minimum distance does not change for codes with even or odd minimum distance which is because we remove an information bit which is zero and therefore minimum distance does not change.

Question 3

We know that if $x[n]$ is real, then $X[N - k] = X^*[k]$.

So, $X[k].X^*[k] = X[k].X[N - k]$.

Also, we know that DFT is periodic with period N . so, $X[N - k] = X[-k]$.

we know inverse DFT of $H[k]X[k]$ is $\sum_{m=0}^{N-1} h[m]x[n - m]$, $-(N - 1) \leq n \leq N - 1$.

So, we have:

$$X[k].X[-k] \Rightarrow \sum_{m=0}^{N-1} x[m]x[m - n] , -(N - 1) \leq n \leq N - 1 = R_n$$